

DSA Lecture 3: Basic Sorting Algorithms (Bubble, Selection, Insertion)

Sorting algorithms are fundamental in computer science, and they rearrange elements of a list or array into a specific order (typically ascending or descending). **Bubble Sort**, **Selection Sort**, and **Insertion Sort** are foundational comparison-based algorithms often taught first due to their simplicity, though they are not the most efficient for large datasets. This lecture will cover the logic and provide C code examples for each.

Before covering sorting algorithms we need to understand Time complexity.

1. Time Complexity

Time complexity is a core concept in Data Structures and Algorithms (DSA) that describes how the running time of an algorithm grows as the size of the input (usually denoted as n) increases. It doesn't measure the actual time in seconds, but rather the rate of growth of the number of operations performed by the algorithm.

Time complexity is expressed using **Asymptotic Notations**, which describe the limiting behavior of a function (the algorithm's running time) as the input size approaches infinity. The most commonly used notation is **Big O** (pronounced "Big Oh").

Common Time Complexities

Here is a breakdown of the most common Big O complexities, from fastest (best) to slowest (worst):

Notation	Name	Rate of Growth	Example in DSA	Analogy
$O(1)$	Constant	Execution time never changes, regardless of n .	Accessing an element in an array by its index.	Checking the first card in a deck.
$O(\log n)$	Logarithmic	Time grows very slowly; the size of the problem is cut down in each step.	Binary Search.	Finding a word in a dictionary (you cut the pages in half each time).
$O(n)$	Linear	Time grows directly proportional to n .	Traversing a list/array (checking every item once).	Walking along every house on a street.
$O(n \log n)$	Linearithmic	Time grows slightly faster than linear, highly efficient for sorting.	Merge Sort, Heap Sort.	A sophisticated delivery system that optimizes routes.
$O(n^2)$	Quadratic	Time grows as the square of n . Becomes very slow for large n .	Nested loops, Bubble Sort, Selection Sort.	Every person in a room shakes hands with every other person.
$O(2^n)$	Exponential	Time doubles with every addition to n . Very slow, only feasible for tiny n .	Certain recursive problems (e.g., traveling salesman brute force).	A nuclear chain reaction.

Rules for Calculating Big O

When analyzing code, we focus on the part of the expression that grows the fastest and drop constants and lower-order terms.

1. **Drop Constants:** $O(2n)$ is $O(n)$.
2. **Drop Lower-Order Terms:** $O(n^2+n)$ is $O(n^2)$. As n grows large, n^2 dominates n .

Code Structure	Complexity
Single Operation: <code>a = b + c;</code>	$O(1)$
Single Loop: <code>for (i=0; i<n; i++) { ... }</code>	$O(n)$
Nested Loop: <code>for (i=0; i<n; i++) { for (j=0; j<n; j++) { ... } }</code>	$O(n^2)$
Dividing Work: <code>while (n > 1) { n = n / 2; }</code>	$O(\log n)$

2. Bubble Sort: The Simple Swapper

Bubble Sort is the simplest sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. The name comes from the way smaller (or larger, depending on the sort order) elements "bubble up" to their correct position.

Logic

1. Start with the first element and compare it to the next adjacent element.
2. If the current element is greater than the next element, swap them.
3. Move to the next pair of elements and repeat.
4. After the first pass, the largest element is at the end.
5. Repeat the passes for the remaining unsorted portion of the array.

C Example

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    // Outer loop for passes (n-1 passes needed)
    for (i = 0; i < n - 1; i++) {
        // Inner loop for comparisons and swaps
        // We do j < n - i - 1 because the last i elements are
        already in place
        for (j = 0; j < n - i - 1; j++) {
            // Compare adjacent elements
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Helper function to print the array
void printArray(int arr[], int size) {
    int i;
    for (i = 0; i < size; i++) {
```

```

        printf("%d ", arr[i]);
    }
    printf("\n");
}

/* Example Usage
int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}
*/

```

3. Selection Sort: The Minimalist Finder

Selection Sort works by repeatedly finding the minimum element from the unsorted part and putting it at the beginning of the array. The array is conceptually divided into a sorted and an unsorted subarray.

Logic

1. Iterate through the array from the first element up to the second-to-last element (the sorted section).
2. In each iteration, assume the current element is the **minimum**.
3. Scan the **remaining unsorted subarray** to find the true minimum element.
4. **Swap** the true minimum element found in the unsorted subarray with the element at the current position of the outer loop.
5. The first elements are now sorted.

C Example

```

#include <stdio.h>

void selectionSort(int arr[], int n) {
    int i, j, min_idx, temp;

    // One by one move boundary of unsorted subarray
    for (i = 0; i < n - 1; i++) {

```

```

        // Find the minimum element in unsorted array
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }

        // Swap the found minimum element with the first element of
        the unsorted part (arr[i])
        if (min_idx != i) {
            temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx] = temp;
        }
    }
}

// (printArray function from Bubble Sort can be reused)

/* Example Usage
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);
    selectionSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}
*/

```

4. Insertion Sort: The Card Player

Insertion Sort builds the final sorted array one item at a time. It iterates through the input elements and consumes one input element in each iteration, growing a sorted output list. Each element is **inserted** into its correct position in the sorted part of the array.

Logic

1. Start from the second element () and consider it the **key**. The first element () is considered a sorted subarray of size 1.

2. Compare the **key** with the elements in the sorted subarray to its left.
3. If a larger element is found in the sorted subarray, shift it one position to the right.
4. Continue shifting until the correct position for the **key** is found (where the previous element is smaller or the beginning of the array is reached).
5. Insert the **key** into that position.
6. Repeat for the next element.

C Example

```
#include <stdio.h>

void insertionSort(int arr[], int n) {
    int i, key, j;
    // Start from the second element (i=1)
    for (i = 1; i < n; i++) {
        key = arr[i]; // Element to be inserted
        j = i - 1;    // Start comparing with the last element of the
sorted subarray

        /* Move elements of arr[0..i-1], that are
           greater than key, to one position ahead
           of their current position */
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key; // Insert the key into its correct position
    }
}

// (printArray function from Bubble Sort can be reused)

/* Example Usage
int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);
    insertionSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}
*/
```

Comparison Summary

Algorithm	Worst Case Time Complexity	Best Case Time Complexity	Stability	In-Place
Bubble Sort	$O(n^2)$	$O(n)$ (If optimized with a swap flag)	Stable	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	Not Stable	Yes
Insertion Sort	$O(n^2)$	$O(n)$ (When array is already sorted)	Stable	Yes

- $O(n^2)$ indicates that as the size of the array () increases, the execution time grows quadratically. This is slow for large datasets.
- $O(n)$ indicates linear growth, meaning the execution time is directly proportional to the size of the input, which is efficient.
- **Stable** means that if two elements are equal, their relative order in the sorted output remains the same as in the input.
- **In-Place** means that the algorithm uses a small, constant amount of extra memory space.